Abshaar: Step-by-Step Implementation Guide

Rauf Nawaz

May 23, 2026

Contents

1 Purpose of This Guide

This guide explains how to build Abshaar from an empty repository into a working open-source system for translating and explaining Punjabi, Urdu, Persian- influenced, Braj, Bhakti, and Sufi poetry in English. It is intentionally granular. The goal is to make the project understandable for future contributors who may care deeply about poetry but may not be machine-learning engineers.

The project should be built in this order:

1. Build the corpus and annotation structure.
2. Create a small gold-standard dataset by hand.
3. Build a local AI-assisted draft pipeline.
4. Add retrieval over glossary, context, and prior reviewed examples.
5. Build a static public website.
6. Add a review loop.
7. Fine-tune only after enough reviewed examples exist.
8. Add chatbot features only when the archive itself is strong.

1.1 Core Principle

Do not begin by training a model. Begin by building a careful archive. The model will only become useful if the project contains strong source text, source metadata, translations, glossary entries, review notes, and poet-specific context.

2 What You Are Building

Abshaar should have three connected layers.

2.1 Layer 1: Archive

The archive stores the original poem, transliteration, source information, translation, explanation, glossary, poet context, timeline events, and review status.

2.2 Layer 2: Local AI Assistant

The AI assistant drafts literal translations, literary translations, tashreeh, glossary notes, and question-answer responses. It must retrieve relevant context before answering. It must never replace human review.

2.3 Layer 3: Public Website

The website lets readers explore poets, poems, timelines, themes, glossary terms, and beginner-friendly explanations. It should run for free on GitHub Pages in the first public version.

3 High-Level Timeline

Phase	Time	Outcome
0	1–2 weeks	Choose one poet, source rules, licensing, and annotation standards.
1	2–4 weeks	Build a small corpus of 20–50 works with source metadata.
2	3–5 weeks	Build local draft pipeline using existing open models.
3	Ongoing	Store human corrections as structured review data.
4	2 weeks	Create evaluation set and scoring rubric.
5	After 500+ reviews	Try LoRA or QLoRA fine-tuning.
6	3–6 weeks	Build static website MVP.
7	After website MVP	Add contribution workflow and public review process.
8	Long term	Expand poets, themes, dictionary, audio, and optional chatbot.

4 Phase 0: Project Decisions

4.1 Step 0.1: Choose the First Poet

Choose one poet for the first version. Do not start with all Sufi and Bhakti poetry at once.

Recommended first poet options:

- Kabir: rich interpretive tradition, many public-domain materials, strong cross-religious vocabulary.
- Bulleh Shah: central for Punjabi Sufi thought, strong theme of ishq, murshid, social critique, and performative tradition.
- Baba Farid: important for Punjabi, Sikh scripture context, and Sufi vocabulary.
- Shah Hussain: strong for Punjabi kafi form and mystical love.
- Waris Shah: larger literary project, better after the system matures.

Recommended first choice for Abshaar: Bulleh Shah. He fits the project's Punjabi, Urdu, Persianate, and Sufi focus especially well. His poetry also forces the system to handle the core problem: literal translation is not enough because words such as ishq, murshid, nafs, haq, and kafir can shift meaning depending on poetic, spiritual, and social context.

Begin with 20–50 short kafis or selected stanzas. Do not begin with a large complete collected works edition. The first release should show quality, not scale.

4.2 Step 0.2: Decide the First Release Scope

Write down the first release scope in a file called `docs/mvp_scope.md`.

Include:

- selected poet;
- number of works;
- allowed source texts;
- scripts to preserve;
- transliteration standard;
- translation style;
- explanation style;
- what is explicitly out of scope.

Example:

```
MVP poet: Bulleh Shah
Works: 30 short couplets or songs
Scripts: Shahmukhi/Perso-Arabic where available, Gurmukhi or other source
scripts if using a licensed source, plus Latin transliteration
Translation types: literal gloss, literary translation, tashreeh
Website: static pages only, no public chatbot yet
Model: local Qwen3 through Ollama for draft explanations
Review: all publishable outputs require human approval
```

4.3 Step 0.3: Decide Licensing

Use separate licenses for code and content.

- Code: MIT or Apache-2.0.
- Project documentation: CC BY 4.0.
- Your translations, tashreeh, glossary, and annotations: CC BY-SA 4.0.
- Source texts: keep their own license and source status.

Important: do not assume that modern translations, recordings, transcriptions, or website annotations are free to reuse. Classical poetry may be public domain, but a modern edition or translation may still be copyrighted.

4.4 Step 0.4: Define the Translation Philosophy

Create a short translation style guide. The model and contributors need to know what "good" means.

Recommended rules:

1. Preserve the image before explaining the abstraction.
2. Keep literal translation separate from literary translation.
3. Keep tashreeh separate from translation.
4. Preserve culturally loaded terms when English would flatten them.
5. Use footnotes or glossary entries instead of forcing everything into the English line.
6. Mark uncertainty and alternate readings.
7. Do not collapse Sufi, Bhakti, Sikh, Islamic, Hindu, folk, and regional concepts into one generic mystical vocabulary.

5 Phase 1: Repository and Local Setup

5.1 Step 1.1: Confirm Git Is Working

From PowerShell:

```
cd "D:\Harvard\Poetry Model Project"
git status --short --branch
```

If the folder is not a Git repository, initialize it:

```
git init
git branch -M main
```

If the remote is not set:

```
git remote add origin https://github.com/RaufNawaz/Abshaar.git
```

5.2 Step 1.2: Keep a Clean Folder Structure

Use this structure:

```
docs/
  01_project_roadmap.md
  02_model_strategy.md
  03_data_and_annotation_guide.md
  04_website_architecture.md
  05_local_setup.md
  06_open_source_governance.md
  07_step_by_step_implementation_guide.tex
```

```
data/
  raw/
    public/
    private/
  processed/
  annotations/
  lexicon/
  context/
  cache/

scripts/
  ingest/
  translate/
  review/
  export/

src/
  abshaar/
    __init__.py
    schemas.py
    ingest.py
    translate.py
    retrieve.py
    review.py
    export_site.py

website/
```

Do not put model files, private scans, or generated caches into Git.

5.3 Step 1.3: Install Required Software

Install these:

- Git;
- Python 3.11 or 3.12;
- Node.js LTS;
- VS Code;
- Ollama;
- optional: WSL2 with Ubuntu for heavier ML work.

5.4 Step 1.4: Create Python Environment

```
cd "D:\Harvard\Poetry Model Project"
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Install minimal dependencies first:

```
pip install pydantic pandas rich
```

Add ML dependencies when needed:

```
pip install torch transformers sentence-transformers accelerate
pip install chromadb fastapi uvicorn
```

Add fine-tuning tools later:

```
pip install peft datasets evaluate
```

6 Phase 2: Build the Corpus MVP

6.1 Step 2.1: Choose Safe Source Texts

For each source, record:

- title;
- editor or translator if applicable;
- publication date;
- URL or bibliographic reference;
- copyright status;
- whether it may be used for training;
- whether it may be republished on the website.

Create `data/context/sources.jsonl`. Each line should look like:

```
{
  "id": "source_0001",
  "title": "Source title",
  "author_or_editor": "Name",
  "year": "1910",
  "url": "https://example.org",
  "rights_status": "public-domain",
  "allowed_uses": ["quote", "train", "republish"],
  "notes": "Why this is safe to use."
}
```

6.2 Step 2.2: Create the Poem Schema

Create `data/processed/poems.jsonl`. Each line is one poem record:

```
{
  "id": "kabir_0001",
  "poet_id": "kabir",
  "title": "First line or working title",
  "work_type": "couplet",
  "source_ids": ["source_0001"],
  "original": {
    "text": "Original text here",
    "script": "Devanagari",
    "language_spans": [
      {
        "text": "Original span",
        "language": "Braj",
        "confidence": 0.8
      }
    ]
  },
  "transliteration": {
    "scheme": "project-latin-v1",
    "text": "Transliteration here"
  },
  "translations": [],
  "tashreeh": [],
  "glossary_terms": [],
  "themes": [],
  "review_status": "draft"
}
```

6.3 Step 2.3: Start With 20 Works

For each work:

1. Add original text.
2. Add source ID.
3. Add transliteration.
4. Add a literal gloss.
5. Add your first literary translation.
6. Add tashreeh.
7. Add glossary terms.
8. Add themes.

9. Mark review status.

Do not let the model create the gold dataset alone. The first 20 works should be human-led. This gives the system a standard to imitate later.

6.4 Step 2.4: Build the Lexicon

Create `data/lexicon/terms.jsonl`. Every term should be poet-specific when needed.

```
{
  "id": "term_ishq_bulleh_shah",
  "headword": "ishq",
  "transliteration": "ishq",
  "languages": ["Persian", "Urdu", "Punjabi"],
  "basic_meaning": "love",
  "poet_specific_meaning": "Divine longing that dissolves ego and rank.",
  "do_not_flatten_to": ["romantic love"],
  "related_terms": ["fana", "murshid", "haqiqat"],
  "example_poems": ["bulleh_0001"],
  "source_ids": ["source_0001"],
  "review_status": "draft"
}
```

Add a field for terms that should remain untranslated. Example: “naam”, “shabad”, “murshid”, “haq”, “fana”, “ishq”, “virah”. Sometimes the right English move is to preserve the original word and explain it.

6.5 Step 2.5: Build Poet Context

Create:

- `data/context/people.jsonl`;
- `data/context/events.jsonl`;
- `data/context/themes.jsonl`.

For timeline events:

```
{
  "id": "event_kabir_0001",
  "poet_id": "kabir",
  "date_label": "15th century",
  "date_start": null,
  "date_end": null,
  "event_type": "historical_context",
  "title": "North Indian devotional and Sufi milieu",
  "description": "Short, sourced note.",
  "source_ids": ["source_0002"],
  "confidence": 0.6
}
```

Use confidence because many classical poet biographies are uncertain.

7 Phase 3: Create the Human Review Workflow

7.1 Step 3.1: Define Review Status Values

Use these statuses:

- `draft`;
- `machine_draft`;
- `human_reviewed`;
- `needs_source_check`;
- `needs_language_check`;
- `publishable`;
- `rejected`.

7.2 Step 3.2: Create Review Records

Create `data/annotations/reviews.jsonl`.

```
{
  "id": "review_0001",
  "poem_id": "kabir_0001",
  "reviewer": "Rauf",
  "date": "2026-05-23",
  "scores": {
    "source_fidelity": 4,
    "metaphor_fidelity": 3,
    "poet_specific_context": 3,
    "literary_quality": 4,
    "beginner_clarity": 5,
    "humility_about_uncertainty": 4
  },
  "problems": [
    {
      "type": "missed_context",
      "note": "The image was translated literally but not explained."
    }
  ],
  "corrected_translation": "Corrected translation here.",
  "corrected_tashreeh": "Corrected explanation here.",
  "style_note": "Preserve the metaphor before explaining it."
}
```

7.3 Step 3.3: Make Review a Habit

After every AI-generated output, ask:

1. Did it preserve the concrete image?
2. Did it understand poet-specific vocabulary?
3. Did it invent context?
4. Did it flatten religious differences?
5. Did it make the English readable?
6. Did it explain too much?
7. Did it explain too little?
8. Is it safe to publish?

The review data is the project's most valuable machine-learning asset.

8 Phase 4: Build the First Local AI Pipeline

8.1 Step 4.1: Model Roles

Use different models for different roles.

Task	Recommended Model	Reason
Baseline translation	IndicTrans2	Strong Indic-to-English baseline for supported languages.
Fallback translation	NLLB-200	Broader language coverage, useful as fallback.
Explanation and rewriting	Qwen3 via Ollama	Strong local instruction-following and multilingual ability.
Retrieval embeddings	BGE-M3	Multilingual embeddings for glossary and context retrieval.
Website search	Pagefind or MiniSearch	Static, free, no backend required.

8.2 Step 4.2: Install and Test Ollama

Ollama is the simplest local model runner for the first version. It gives you a local HTTP API, which means the rest of the project can call the model without special cloud credentials.

Install Ollama from:

```
https://ollama.com/
```

After installation, confirm it is running:

```
ollama --version
ollama list
```

If the Ollama background service is running, this URL should respond:

```
http://localhost:11434
```

8.3 Step 4.3: Install the Interpretive LLM

```
ollama pull qwen3:8b
ollama run qwen3:8b
```

If the machine is slow:

```
ollama pull qwen3:4b
```

If the machine is stronger:

```
ollama pull qwen3:14b
```

Use Qwen3 for:

- tashreeh drafting;
- literary translation rewriting;
- glossary explanation;
- alternate readings;
- question-answering over retrieved context;
- self-checking whether an explanation overclaims.

Do not use Qwen3 alone as the translation engine. Use it after baseline translation and retrieval.

8.4 Step 4.4: Call Ollama From Python

Install the Python client:

```
pip install ollama
```

Minimal test:

```
from ollama import chat

response = chat(
    model="qwen3:8b",
    messages=[
        {"role": "user", "content": "Explain what tashreeh means in one sentence."}
    ],
)

print(response["message"]["content"])
```

For project use, wrap this in a function:

```
def ask_llm(system_prompt, user_prompt, model="qwen3:8b"):
    response = chat(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
        options={
            "temperature": 0.3,
            "top_p": 0.9,
        },
    )
    return response["message"]["content"]
```

8.5 Step 4.5: Install Baseline Translation Models

Use two baseline translation routes.

8.5.1 IndicTrans2

Use IndicTrans2 for supported Indic-to-English translation. It is the preferred baseline for supported Indic languages, including Punjabi and Urdu-related coverage where available.

Install dependencies:

```
pip install torch transformers IndicTransToolkit
```

Model to start with:

```
ai4bharat/indictrans2-indic-en-1B
```

Important:

- some Hugging Face model files may require accepting terms on the model page;
- do not commit model weights to Git;
- use this model for baseline meaning, not final literary translation;
- for poetry, line segmentation matters.

8.5.2 NLLB-200

Use NLLB-200 as a fallback when IndicTrans2 does not handle the source well.

Install:

```
pip install sentencepiece sacremoses
```

Model:

```
facebook/nllb-200-distilled-600M
```

Use NLLB for:

- fallback baseline translation;
- comparison against IndicTrans2;
- broader multilingual snippets;
- debugging whether a word/phrase is being lost.

8.6 Step 4.6: Install Embedding and Retrieval Models

Use BGE-M3 for multilingual retrieval.

```
pip install sentence-transformers chromadb
```

Minimal embedding test:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-m3")
texts = [
    "Ishq in Bulleh Shah often points to divine love and ego-dissolution.",
    "Murshid refers to the spiritual guide in Sufi contexts."
]
embeddings = model.encode(texts)
print(embeddings.shape)
```

For the first version, store embeddings in Chroma as a generated cache. Keep the canonical source data in JSONL.

8.7 Step 4.7: How the Models Work Together

The project should use the models in a pipeline, not as isolated chatbots.

1. Load one Bulleh Shah poem record.
2. Detect script and segment the poem into lines.
3. Run IndicTrans2 for baseline translation if the language/script is supported.
4. Run NLLB-200 as a fallback or comparison.
5. Retrieve glossary entries using exact match and BGE-M3 semantic search.
6. Retrieve poet context, themes, and reviewed examples.
7. Send the original, transliteration, baseline translations, and retrieved context to Qwen3.
8. Ask Qwen3 for structured output: literal gloss, literary translation, tashreeh, key terms, alternate readings, and uncertainty.
9. Save the raw output.
10. Review manually.
11. Store the corrected version as the new gold example.

8.8 Step 4.8: Recommended Python Modules

Create these files later:

```
src/abshaar/schemas.py
src/abshaar/io.py
src/abshaar/translate_baseline.py
src/abshaar/embed.py
src/abshaar/retrieve.py
src/abshaar/generate.py
src/abshaar/review.py
src/abshaar/export_site.py
```

Responsibilities:

- `schemas.py`: Pydantic models for poems, terms, sources, reviews.
- `io.py`: read and write JSONL.
- `translate_baseline.py`: IndicTrans2 and NLLB wrappers.
- `embed.py`: BGE-M3 embedding generation.
- `retrieve.py`: exact match plus vector retrieval.
- `generate.py`: Qwen3 prompts through Ollama.
- `review.py`: save human corrections.
- `export_site.py`: export clean public data for the website.

8.9 Step 4.9: Draft Pipeline Steps

The first pipeline should run like this:

1. Load one poem from `poems.jsonl`.
2. Segment the poem into lines or couplets.
3. Detect script and probable language spans.
4. Create or load transliteration.
5. Run baseline translation.
6. Retrieve glossary entries and poet context.
7. Ask the LLM to produce a structured draft.
8. Save the model output.
9. Ask a human reviewer to correct it.
10. Save the review.

8.10 Step 4.10: Required Output Format

The model should return JSON-like structured data:

```
{
  "literal_gloss": "...",
  "direct_translation": "...",
  "literary_translation": "...",
  "tashreeh_beginner": "...",
  "tashreeh_advanced": "...",
  "key_terms": [
    {
      "term": "...",
      "meaning_in_context": "...",
      "uncertainty": "low/medium/high"
    }
  ],
  "alternate_readings": ["..."],
  "uncertainty_notes": ["..."],
  "retrieved_context_used": ["context_id_1", "term_id_2"]
}
```

If a model cannot return valid JSON reliably, store the raw output and parse it later. Do not lose raw outputs; they are useful for debugging.

8.11 Step 4.11: Prompt Template

Use a staged prompt, not a vague request.

```
You are assisting with a scholarly but beginner-friendly translation
of classical South Asian mystical poetry.

Rules:
1. Preserve the original image before explaining it.
2. Separate literal gloss, literary translation, and tashreeh.
3. Do not invent biography or doctrine.
4. Use retrieved context only when relevant.
5. Mark uncertainty and alternate readings.
6. If a term should remain untranslated, say so.

Poet:
{poet_name}

Original:
{original_text}

Transliteration:
{transliteration}
```

```
Known glossary:
{retrieved_glossary}

Poet context:
{retrieved_context}

Prior reviewed examples:
{few_shot_examples}

Return:
- literal_gloss
- direct_translation
- literary_translation
- beginner_tashreeh
- advanced_tashreeh
- key_terms
- alternate_readings
- uncertainty_notes
```

9 Phase 4A: Initial Dataset Template for Bulleh Shah

9.1 Why Bulleh Shah First

Bulleh Shah is the recommended first poet because he is directly aligned with the project's goals:

- Punjabi/Sufi center of gravity;
- Persian and Islamic vocabulary woven into Punjabi poetic expression;
- strong metaphysical concepts that cannot be translated literally;
- social critique, religious critique, love, ego, and spiritual authority;
- strong usefulness for a future public glossary.

9.2 Initial Dataset Files

Create template files under:

`data/templates/`

Then copy them into the real data folders when you begin:

```
data/templates/poems.template.jsonl        -> data/processed/poems.jsonl
data/templates/terms.template.jsonl        -> data/lexicon/terms.jsonl
data/templates/sources.template.jsonl      -> data/context/sources.jsonl
data/templates/people.template.jsonl       -> data/context/people.jsonl
data/templates/events.template.jsonl       -> data/context/events.jsonl
data/templates/themes.template.jsonl       -> data/context/themes.jsonl
data/templates/reviews.template.jsonl      -> data/annotations/reviews.jsonl
data/templates/model_outputs.template.jsonl -> data/annotations/model_outputs.jsonl
```

9.3 Minimum Fields for Every Bulleh Shah Poem

Every poem record should include:

- stable ID, such as `bulleh_shah_0001`;
- source IDs;
- title or first line;
- original text;
- script;
- language spans;
- transliteration;
- segmentation;
- literal gloss;
- literary translation;
- tashreeh;
- key terms;
- themes;
- review status;
- rights status.

9.4 Starter Glossary Terms for Bulleh Shah

Begin with terms like:

- ishq;
- murshid;
- pir;
- haq;
- nafs;
- fana;
- kafir;
- masjid;
- mandir;
- yaar;

- rang;
- faqir.

Each term should have a basic meaning and a Bulleh Shah-specific meaning. The poet-specific meaning is what will make the model useful.

10 Phase 5: Retrieval-Augmented Generation

10.1 Step 5.1: What Should Be Retrieved

For each poem, retrieve:

- glossary terms that appear in the poem;
- poet-specific notes;
- related reviewed poems;
- theme notes;
- timeline events relevant to the poem;
- source notes.

10.2 Step 5.2: Create Embedding Documents

Transform records into small searchable passages.

Example glossary embedding document:

```
ID: term_ishq_bulleh_shah
TYPE: glossary
TEXT:
For Bulleh Shah, ishq often means divine longing and transformative love
that challenges social hierarchy, ego, and formal identity. It should not
be reduced to romantic love without context.
```

Example timeline embedding document:

```
ID: event_kabir_0001
TYPE: event
TEXT:
Kabir is associated with a North Indian devotional world in which Hindu,
Islamic, artisan, and oral traditions intersect. Biographical certainty is
limited, so claims should be presented carefully.
```

10.3 Step 5.3: Retrieval Rules

For every model answer:

1. retrieve at least three relevant context chunks;
2. pass them into the prompt;
3. ask the model to cite which chunks it used;
4. reject answers that make claims not grounded in the source or marked as interpretation;
5. store retrieval IDs with the generated output.

11 Phase 6: Evaluation Before Fine-Tuning

11.1 Step 6.1: Build an Evaluation Set

Create 50–100 passages that are held out from normal prompting. These should not be repeatedly used as few-shot examples.

Each evaluation item should include:

- original text;
- transliteration;
- human literal gloss;
- human literary translation;
- human tashreeh;
- key terms;
- accepted alternate readings;
- source IDs.

11.2 Step 6.2: Scoring Rubric

Score every model output from 1 to 5:

Category	Question
Source fidelity	Does it preserve what the original says?
Image fidelity	Does it preserve the metaphor before abstracting it?
Poet-specific context	Does it use this poet's meaning of key terms?
Cultural grounding	Does it explain context without overclaiming?
Literary quality	Does the English feel alive, not mechanical?
Beginner clarity	Can a new reader understand the stakes?
Uncertainty	Does it mark ambiguity and avoid false certainty?

Citation discipline	Does it stay grounded in source and retrieved context?

11.3 Step 6.3: Decide If Fine-Tuning Is Worth It

Do not fine-tune until:

- you have at least 500 reviewed examples;
- the prompt-and-retrieval system is already working;
- you know exactly what behavior needs improvement;
- you have a held-out evaluation set;
- you can compare pre-fine-tune and post-fine-tune outputs.

12 Phase 7: Fine-Tuning Plan

12.1 Step 7.1: Fine-Tune the Right Thing

Fine-tuning should teach:

- output format;
- translation style;
- explanation style;
- how to preserve terms;
- how to mark uncertainty;
- how to avoid flattening metaphors.

Fine-tuning should not be the main storage place for knowledge. Knowledge should stay in the corpus, glossary, and retrieval system.

12.2 Step 7.2: Training Data Format

Use instruction-response records:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You translate and explain classical mystical poetry..."
    },
    {
      "role": "user",
      "content": "Poet: Kabir\nOriginal: ...\nContext: ..."
    },
    {
```

```
      "role": "assistant",
      "content": "{ reviewed JSON output here }"
    }
  ]
}
```

12.3 Step 7.3: LoRA or QLoRA

Use LoRA or QLoRA rather than full fine-tuning.

Recommended progression:

1. Try prompt-only baseline.
2. Try prompt plus retrieval.
3. Try few-shot examples.
4. Try LoRA on 500–2,000 examples.
5. Try preference tuning only after collecting rejected and corrected pairs.

12.4 Step 7.4: Training Safety

Before releasing a fine-tuned model:

- confirm training data rights;
- remove private notes;
- remove copyrighted modern translations unless licensed;
- include a model card;
- disclose base model;
- disclose intended use and limitations;
- explain that outputs require human review.

13 Phase 8: Website Development

13.1 Step 8.1: Start Static

Use Astro for the first website.

```
npm create astro@latest website
cd website
npm install
npm run dev
```

Use a static site because:

- it can be hosted on GitHub Pages for free;

- it is safer and easier to maintain;
- contributors can edit content without managing servers;
- it keeps the archive public even if no AI backend is running.

13.2 Step 8.2: Website Pages

Build pages in this order:

1. Home page with search and featured poet.
2. Poet profile page.
3. Works index.
4. Poem page.
5. Glossary page.
6. Theme page.
7. Timeline page.
8. Source and credits page.
9. Contribute page.
10. Ask page with curated FAQ.

13.3 Step 8.3: Poem Page Requirements

Each poem page should show:

- original text;
- transliteration;
- literal gloss;
- literary translation;
- beginner tashreeh;
- advanced notes if available;
- glossary highlights;
- themes;
- related works;
- source and license;
- review status.

13.4 Step 8.4: Timeline Requirements

Timeline entries should distinguish:

- life event;
- uncertain biography;
- historical context;
- composition or oral tradition;
- manuscript or publication;
- later reception;
- modern performance or interpretation.

Use confidence labels. Classical poet timelines are often uncertain, and the site should not pretend otherwise.

13.5 Step 8.5: Chatbot Options

Do not make the chatbot the first public feature. Build search and curated FAQ first.

Options:

1. Static FAQ and search: best first release.
2. Browser-local AI through WebLLM: free but hardware-dependent.
3. Separate FastAPI backend with Ollama: best quality but requires hosting.

GitHub Pages alone cannot host a normal server-side chatbot.

14 Phase 9: Public Contribution Workflow

14.1 Step 9.1: Contributor Paths

Allow contributors to help without understanding the whole system.

Contribution types:

- source correction;
- transliteration correction;
- glossary addition;
- alternate translation;
- tashreeh improvement;
- timeline event;
- theme note;
- website bug fix.

14.2 Step 9.2: Contribution Template

Every content contribution should answer:

1. What are you changing?
2. What source supports it?
3. What is the rights status?
4. Is there uncertainty?
5. Does this affect published pages?
6. Does this require language review?
7. Does this require interpretation review?

14.3 Step 9.3: Review Roles

Use these roles:

- maintainer;
- language reviewer;
- interpretation reviewer;
- source/license reviewer;
- technical reviewer.

  At the beginning, one person may hold several roles. Over time, separate them.

15 Detailed Weekly Plan

15.1 Week 1

- Finalize MVP poet.
- Create `docs/mvp_scope.md`.
- Create source rules.
- Create translation style guide.
- Confirm licensing.
- Confirm GitHub remote and branch workflow.

15.2 Week 2

- Collect 20 safe source texts.
- Create `sources.jsonl`.
- Create first `poems.jsonl` records.
- Add transliterations.
- Add source notes.

15.3 Week 3

- Write literal glosses for 10 poems.
- Write literary translations for 10 poems.
- Write beginner tashreeh for 10 poems.
- Add 20 glossary terms.
- Add 10 theme records.

15.4 Week 4

- Finish 20-poem gold set.
- Create review rubric.
- Create review JSONL records.
- Install Ollama and Qwen3.
- Test one local prompt manually.

15.5 Weeks 5–6

- Build Python schemas with Pydantic.
- Build JSONL loader.
- Build first draft generation script.
- Save raw model outputs.
- Review 10 machine drafts.

15.6 Weeks 7–8

- Add BGE-M3 embeddings.
- Build retrieval over glossary and context.
- Add retrieved context to prompts.
- Compare no-retrieval vs retrieval outputs.
- Improve prompt template.

15.7 Weeks 9–10

- Create website project.
- Build poem page template.
- Build poet page.
- Build glossary page.
- Export JSON data for website.

15.8 Weeks 11–12

- Build timeline page.
- Add search.
- Add source and credits page.
- Add contribution page.
- Deploy to GitHub Pages.

16 First MVP Definition of Done

The MVP is done when:

- one poet has at least 20 reviewed works;
- every work has original text, source, transliteration, translation, and tashreeh;
- at least 20 glossary terms exist;
- at least 10 timeline events exist;
- the local AI draft script works for one poem;
- generated drafts are saved and reviewable;
- the static website shows poet, works, poem pages, glossary, and sources;
- no private or copyrighted material is committed accidentally.

17 Common Mistakes to Avoid

- Starting with model training before building the corpus.
- Scraping modern translations without permission.
- Treating one English word as the final meaning of a mystical term.
- Mixing literal translation and tashreeh into one output.
- Letting the model invent biographical context.

- Building a chatbot before the archive is useful.
- Committing large model files to Git.
- Publishing uncertain claims without confidence labels.
- Making the website depend on a paid backend from the start.

18 Recommended Source Links

- IndicTrans2: https://github.com/AI4Bharat/IndicTrans2
- IndicTrans2 model card: https://huggingface.co/ai4bharat/indictrans2-indic-en-1B
- NLLB documentation: https://huggingface.co/docs/transformers/en/model_doc/nllb
- Qwen3 model card: https://huggingface.co/Qwen/Qwen3-8B
- Ollama Qwen3: https://ollama.com/library/qwen3
- BGE-M3 embeddings: https://huggingface.co/BAAI/bge-m3
- WebLLM: https://github.com/mlc-ai/web-llm
- PEFT LoRA documentation: https://huggingface.co/docs/peft/en/developer_guides/lora
- Axolotl: https://docs.axolotl.ai/
- Ajab Shahar reference: https://ajabshahar.com/map

19 Final Strategic Recommendation

Build Abshaar as a public archive with AI assistance, not as a model-only project. The most sustainable version is:

1. public data and documentation;
2. transparent human review;
3. local open-weight models;
4. static website for free access;
5. optional local AI for advanced users;
6. fine-tuned models only after the corpus is strong.

This preserves the poetry, keeps the project open, and makes the work accessible to readers who do not already know the languages or traditions.